

Deception at the Registry Layer

An Architecture for Detecting Autonomous AI Agents in Model Context Protocol Environments

Author: Harshad Sadashiv Kadam

Version: 1.2 · September 2026

License: CC BY 4.0

This is independent research, conducted on the author's personal infrastructure and personal Cloudflare account. It does not draw on any employer's systems, data, or internal tooling, and reflects the author's own views.

Abstract

Autonomous AI agents now explore enterprise systems the way a human intruder would — enumerating tools, probing capabilities, and reaching for credentials — but at machine speed and without the deviation that anomaly-based detection is built to catch. Enumeration is not tradecraft for an agent; it is the default behavior. When touching everything reachable is the workload, "touched a lot of things" is not an anomaly to alert on.

Deception closes this gap, and the security field has converged on that conclusion. A decoy no legitimate process should ever touch produces a clean, baseline-free signal: the touch is the alert. The open question is no longer *whether* to deceive agents, but *how to structure* deception so the resulting signal is high-fidelity and hard to evade.

This paper argues that the unit of deception should be the **registry**, not the individual token. It presents an architecture in which real, production Model Context Protocol (MCP) servers and decoy MCP servers sit side by side behind a single gateway portal, indistinguishable to an enumerating agent, with a two-stage detection boundary that separates an agent *finding* a credential from an agent *using* it. The architecture is serverless, deployable on edge infrastructure with zero standing servers, and was demonstrated live — real and decoy MCP servers behind one portal, an agent enumerating them as equally valid, and a canary firing on privileged-tool use — at the OWASP 25th Anniversary Conference in February 2026.

1. The problem: intent is invisible at the API layer

Enterprises increasingly wire autonomous agents into their workflows. Through the Model Context Protocol, an agent can discover a catalog of tools and invoke them without a human directing each step. That is the feature. It is also the flaw.

In an MCP environment, an agent probing your infrastructure looks identical, at the API layer, to one running a sanctioned workflow. Two agents can make the same call to the same tool — one doing legitimate work, one mapping what it can reach. From the outside there is no difference. Security systems built on request inspection assume malicious behavior *looks* different. Against an autonomous agent it does not.

This produces a classification ceiling. An agent can enumerate tools, test capabilities, and probe for credentials while every control in the stack reports normal activity. The intent is invisible, and intent is the only thing that distinguishes reconnaissance from work.

The scale is not hypothetical. MCP adoption grew from roughly 100,000 downloads in late 2024 to over 8 million by April 2025, surpassing 97 million monthly SDK downloads by early 2026, with deployments across major enterprises exposing internal tools, APIs, and credentials to agent-driven access. The protocol was designed for interoperability, not for exposing or detecting agent intent.

2. Why anomaly detection fails here

Anomaly-based detection — SIEM correlation, EDR behavioral baselines, thresholds — assumes attackers move selectively: a target, a path, an effort to stay quiet. Deviation from a baseline is the signal. That logic holds for human attackers.

Autonomous agents do not move selectively. Enumeration is their baseline. They touch everything reachable, at machine speed, because that is how they explore what is available. There is no deviation to alert on when exhaustiveness is the workload.

This inverts the usual relationship between attacker capability and defender advantage. A more capable model enumerates *more* thoroughly, not less. If the detection mechanism depends on the agent touching things, then a better attacker trips it more often. Independent vendor research has since observed this at the cloud-account level: in Tracebit's May 2026 study of ten frontier models attacking an AWS environment, the most capable models tripped more canaries, not fewer. The defense improves as the attacker improves — a property no signature- or baseline-based control has. Whether the same holds at the MCP registry layer is an open question (Section 13).

3. Why single decoys are necessary but not sufficient

A decoy that no legitimate process should ever touch is the cleanest signal in security: no baseline, no threshold, no false-positive rate to manage. The touch is the alert. The field has, correctly, converged on deception as a core detection layer for agentic threats, and drop-in decoy tokens — a single fake MCP server, a single decoy tool added to an agent's configuration — are an excellent first move. They are cheap, fast to deploy, and require no infrastructure.

But a single decoy has two limitations that matter as agentic attacks become routine.

First, a lone decoy can be avoided. A single fake server that looks unusually tempting is a single thing for a careful agent to learn to route around. The decoy's value depends on the agent not being able to tell it apart from the real environment — and a standalone decoy sits *beside* the real environment rather than *within* it.

Second, a single interaction cannot separate curiosity from intent. A conversational decoy that asks a caught agent to identify itself receives an answer — but the answer may be fabricated. Publicly demonstrated conversational decoys have elicited hallucinated responses (for example, an invented SSH host key) from probing agents. For catching *that* the agent is lying, this is fine. But for an alert a SOC will act on, "something answered a question" is a weaker finding than "something executed a privileged action." One exchange tells you something touched the decoy. It does not reliably tell you what it intended, and what it says may be fiction.

The response to both limitations is architectural, not a cleverer single token.

4. How MCP server portals work

The architecture in Section 6 is not abstract — it targets a specific, deployed mechanism: gateway "portals" that centralize multiple MCP servers behind one authenticated endpoint. Cloudflare's MCP server portals are the reference implementation used throughout this paper and in the OWASP demonstration (Section 8). MCP server portals became generally available to all Cloudflare customers on September 24, 2026, after an open beta. This section summarizes their public documented behavior as of that release, because both the attack surface in Section 5 and the architecture in Section 6 follow directly from it.

4.1 Request flow

A portal centralizes multiple MCP servers onto one HTTP endpoint (`https://<subdomain>.<domain>/mcp`). A session proceeds as follows:

1. The MCP client connects to the portal URL and receives a `401` with OAuth discovery metadata.
2. The user authenticates through the platform's identity layer (Cloudflare Access), or a machine presents service-token headers.
3. The identity is validated, and the portal returns the combined tool catalog from every enabled upstream server.

4. On a tool call, the portal resolves the target upstream server from the tool's namespace, attaches the appropriate credential, and proxies the request. Gateway routing, where enabled, adds HTTP logging and inspection on this path.
5. The upstream server responds through the same path.

4.2 Naming

Every tool and prompt exposed through a portal is prefixed with its server ID:

`{server_id}_{original_name}`. A tool `list_issues` on a server with ID `github` appears as `github_list_issues`. Admins can rename tools and rewrite descriptions at the server or portal level; clients only ever see the rewritten version. This namespacing is what makes a decoy server's tools visually consistent with real ones (Section 5, under *Reconnaissance*) — and, in the wrong hands, what makes control-plane tool-description rewriting possible at all (Section 5, under *Manipulation and persistence*).

4.3 Built-in portal tools

Every portal exposes tools of its own for listing and toggling upstream servers, and — with context optimization enabled — a regex search tool across tool names, descriptions, and schemas. These are catalog and search primitives: an enumerating agent uses them for reconnaissance without ever touching an upstream server directly.

Portals also support Code Mode, which replaces the upstream tool definitions with two tools — search and code execution — so that the agent discovers tools by querying for them and calls them from generated code. Admins set a per-portal policy: off, opt-in, on by default, or enforced. Under an enforced policy, no tool definition is sent to the model up front; every tool an agent uses, it first found by search. This changes what makes a decoy discoverable (Section 6.2).

4.4 Credentials and policy

Each upstream server carries its own access policy, visible only to identities that match it. A per-server setting decides whether each user authenticates with their own credential or whether all portal traffic to that server shares one admin credential; machine sessions using service tokens always use the admin credential and are excluded from servers that require per-user auth. This credential-sharing design is the basis of the confused-deputy risk described in Section 5, under *Privilege*.

4.5 Synchronization and logging

Tools and prompts for automatically registered servers re-sync on a fixed interval using the admin credential; new tools discovered during sync are enabled automatically. Background sync does not route through the inspection path used for live traffic. Portal logs record time, status, server, capability, and duration — not call arguments. Logpush export (Enterprise plans) adds identity and session context — user, client IP, session ID, JSON-RPC method, tool name — but its dataset likewise carries no arguments, request bodies, or response content. Both properties matter for detection design: sync introduces a delay and a blind spot (Section 5, under *Evasion*), and thin logging is precisely what a decoy must compensate for (Section 6.3).

5. Threat model: attacking the portal

The attacker here is an autonomous agent, or an operator driving one, that has obtained a foothold: a stolen portal session or service token, a compromised workstation with a connected MCP client, or a credential with control-plane access to the portal itself. The same analysis applies to an authorized researcher testing a portal under permission. The stages below follow the attacker's natural sequence, each grounded in the portal mechanics of Section 4.

Reconnaissance. The full inventory is one call away: the portal's list-servers tool returns every server's ID and name, and the combined tool list leaks internal naming conventions through the `{server_id}_` prefix (Section 4.2). Where regex search over tools is enabled (Section 4.3), an agent can search directly for the capability it wants — credentials, admin functions — instead of reading through the full catalog.

Access. A stolen portal session or service token carries the full view available to that identity. Service-token authentication is positioned for exactly this population — autonomous agents and machine-to-machine connections — so a leaked token is, in practice, an agent identity. Separately, a server blocked by policy for a given user may still be reachable at its direct upstream URL, bypassing the portal (and its logs) entirely, unless the platform's identity layer is configured as that server's own OAuth provider.

Privilege. Where a server does not require per-user auth (Section 4.4), every identity connected through the portal inherits the shared admin credential's access to that server — a confused-deputy pattern documented broadly in the MCP security literature. Service-token sessions always operate this way. Controls that assume a human is present — independent MFA, purpose justification, step-up authentication — do not travel with a request once it is proxied through the portal under a shared credential.

Manipulation and persistence. An identity with control-plane access to the portal can rewrite a tool's name or description at the portal layer without touching the upstream server at all; every connected agent then receives the rewritten version. This is tool poisoning delivered through the registry rather than the server. Separately, because new tools are enabled automatically on sync (Section 4.5), a compromised or malicious upstream server can introduce new capabilities that reach every connected agent within one sync interval.

Evasion. Portal logs record that a tool was called, not what it was called with (Section 4.5) — a defender sees the shape of an interaction, not its content. Background sync and direct-to-upstream connections both bypass the inspected traffic path. And a silently expired admin credential simply removes a server from the portal with no alert, a failure mode indistinguishable from a server disabled for unrelated reasons.

Detection boundary. Below this boundary — enumeration, capability testing, parameter probing, schema inspection — activity is indistinguishable from legitimate exploration and should not, by itself, alert (Section 2). Above it — retrieval of an artifact that no ordinary task requires, followed by an attempt to

use it — intent is unambiguous. The architecture in Section 6 is built to leave that boundary crossing as the only thing that fires.

6. The architecture: deception at the registry layer

The core proposal of this paper is to make the **registry** the unit of deception. Instead of deploying one decoy beside a real environment, blend real and decoy MCP servers into a single gateway registry so that, to an enumerating agent, they are indistinguishable.

6.1 Server-level side-by-side (the core contribution)

An MCP gateway portal presents a catalog of MCP servers to any connecting agent. Behind that single portal sit both **real, production MCP servers** — including third-party servers the organization genuinely uses — and **decoy MCP servers** built for detection, registered through the same workflow and following the same server-ID naming convention (Section 4.2) as their real neighbors. The agent enumerating the portal sees them as one list. There is no "the honeypot" to avoid, because the honeypots are interleaved with the real servers the agent actually wants.

This is the property a standalone decoy cannot provide: the enumeration itself becomes the detection surface. An agent cannot map the environment without touching the registry, and it cannot tell, from the outside, which servers are bait.

Counters: reconnaissance (Section 5). The attacker's first and cheapest step — list the servers — returns decoys mixed with real ones.

(Validated. See Section 8 and the accompanying diagram.)

6.2 Tool-level tiering within a server

Within a decoy server, tools are arranged in tiers of increasing sensitivity, so that escalation is observable as a *path* rather than a single event:

- **Tier 1 — Safe / informational.** Genuinely useful, answerable tools (e.g. listing clusters, reading a namespace quota, requesting access). No detection fires here; legitimate automation touches these constantly, and operational realism is what keeps the environment believable.
- **Tier 2 — Escalation zone.** Infrastructure tools (e.g. service endpoints, CI webhooks) that no ordinary task requires. Intent is beginning to shift. Each returns an infrastructure artifact — an internal hostname, a webhook URL — with a canary embedded in it, so these tools are tripwires in their own right.
- **Tier 3 — Privileged artifact.** A single high-value artifact — a credential or configuration — with a canary embedded inside it.

The detection boundary therefore sits between Tier 1 and Tier 2: every tool past it hands out an artifact that fires in two stages (Section 6.3), and the Tier 3 credential is the strongest of them.

Where a portal runs Code Mode or search-based context optimization (Section 4.3), tools are found by query rather than read from a list. Tier 2 and Tier 3 names and descriptions should therefore contain the terms an attacker would search for — credentials, secrets, admin, kubeconfig — while staying plain and non-directive (Section 7, step 4). A decoy that never matches a search is never found.

6.3 The two-stage detection boundary

Every artifact past the boundary fires in two distinct stages. For the Tier 3 credential:

- **Stage 1 — Access.** The agent *retrieves* the credential. Meaningful, but not yet conclusive; an aggressive scanner might grab it.
- **Stage 2 — Use.** The agent *attempts to use* the credential — a real execution attempt against the target.

Crossing both stages has one explanation. Critically, Stage 2 is not something the agent *says*; it is something it *does*. That makes the two-stage chain resistant to the hallucination problem that undermines single-exchange conversational decoys. The behavioral chain — access followed by use, delivered to a SIEM or alerting channel with the full sequence and the edge logs behind it — is the finding, and it is captured at the decoy itself (full arguments, headers, source, timing), compensating for what portal logs alone do not record (Section 4.5).

Tier 2 artifacts follow the same pattern. Retrieving an endpoint list or a webhook configuration is Stage 1; resolving the embedded hostname (a DNS canary) or calling the webhook URL (an HTTP canary) is Stage 2. The reference decoy, KubeTrap (github.com/harshadk99/mcp-deception-incubator-kubernetes), has run all three surfaces — credential, DNS and webhook — since March 2026. Every call to a Tier 2 or Tier 3 tool also fires a web-bug canary that names the tool and the MCP client, so Stage 1 is attributed before anything is used.

The two records are complementary. Exported portal logs carry identity and session context for every call — user, client IP, session ID, tool name — but no arguments; the decoy carries full arguments but only the identity the portal forwards. Joining them on session and time turns a canary firing into an attributed finding: which identity, from where, crossed which stage.

Counters: privilege and manipulation (Section 5). A confused-deputy grab of the credential is Stage 1; a poisoned-description attack that lures an agent toward using it is caught at Stage 2 regardless of what the description said.

6.4 Zero standing infrastructure

The entire portal and its servers run serverless on edge infrastructure (Cloudflare Workers), with detection provided by open deception primitives (Canarytokens). There are no servers to provision, patch, or scale, which makes the decoy environment cheap to run, globally distributed, and easy to keep believable alongside real production servers.

6.5 Breadcrumbs for the bypass path

Section 5 notes that an attacker who learns upstream URLs can skip the portal entirely. Decoy upstream URLs, placed where an attacker would look for them — configuration files, internal documentation, repositories — extend the tripwire to that path. Where gateway-level MCP traffic detection is available, a direct connection to a decoy host is a second, independent signal alongside the decoy's own logging.

Counters: the direct-URL bypass (Section 5, under Access).

6.6 Canaries on the control plane (proposed)

Two portal behaviors are themselves plausible tripwires, though neither has been demonstrated in the validation reported in Section 8 and both are offered here as future work rather than a proven claim:

- **Definition canaries.** A decoy's tool names and descriptions should never legitimately change. If a sync or a control-plane API call changes them, someone has touched the upstream server or the portal configuration itself.
- **Toggle canaries.** A decoy server left disabled by default. An agent that enables it through the portal's own toggle tool is exploring beyond its assigned tools.

Counters: manipulation and persistence (Section 5).

6.7 What the decoy says back

A decoy's response shapes what happens next, and the right choice depends on the goal: a refusal that mimics a real control firing, sustained engagement with fabricated data to prolong observation, content designed to trip the attacking model's own safety training, or direct interrogation of the agent. Section 3 already covers why an interrogation-style response is a weaker finding on its own than the behavioral chain in Section 6.3 — an agent's self-report may be fabricated, so any response strategy should be treated as supplementary evidence, not the primary signal.

7. Deployment guidance

A working version of this architecture is a small build, not a research project. The steps below assume an existing gateway account and identity provider already in place.

1. **Inventory the real portal.** List server IDs, naming conventions, auth modes, and which servers run without per-user auth (Section 4.4). These choices define what a convincing decoy looks like and where the confused-deputy risk sits.
2. **Build one decoy server.** A remote MCP server on Cloudflare Workers is enough (Section 6.4). Log every request — arguments included — to a webhook or SIEM.
3. **Register it through the normal workflow.** Use a server ID that fits the organization's naming convention (Section 4.2). Attach an access policy that matches the intended visibility: open to everyone for maximum coverage, or scoped to identities that should never touch it, so that any call

is anomalous by construction.

4. **Write the tools in tiers.** Two or three tools per tier (Section 6.2), plain descriptions, no instructions aimed at the model — a decoy that tries to steer an agent is tool poisoning in its own right, and it would confuse benign agents as well as hostile ones. Embed a canary in every Tier 2 and Tier 3 artifact — a hostname, a webhook URL, a credential — so each one fires on retrieval and again on use.
 5. **Route alerts somewhere a human will see them**, with identity context attached. Where available, export portal logs via Logpush to the same SIEM and correlate on session ID (Section 6.3).
 6. **Test with your own agent.** Connect through the portal, confirm the decoy appears alongside real servers, walk it through both stages, and confirm the alert fires with arguments captured.
 7. **Add breadcrumbs** (Section 6.5) for the direct-URL bypass path.
 8. **Review monthly.** Re-check that decoys still sync, still match current naming, and still alert. Silent admin-token expiry (Section 4.5) applies to decoys as much as to real servers; expired servers can be re-authenticated directly from the portal.
-

8. Validation

The architecture was implemented and demonstrated live at the **OWASP 25th Anniversary Conference (February 2026)**, in the session *MCP Deception Incubator — Honeytraps as a Framework for Zero Trust AI Environments*.

The demonstration showed, in sequence:

1. **A single gateway portal** (Cloudflare AI controls / MCP server portal) with four MCP servers registered side by side: two real, production servers (Cloudflare's own hosted `bindings.mcp.cloudflare.com` and `docs.mcp.cloudflare.com`) and two decoy servers built by the author (`mcp-threat-trap` and `mcp-deception-incubator-kubernetes`), all in READY state.
2. **An agent authorization flow** in which a connecting MCP client was offered all four servers together, real and decoy indistinguishable in the same list.
3. **A real agent harness** (Cloudflare Workers AI LLM Playground) pointed at the portal, enumerating the combined tool catalog across real and decoy servers.
4. **A live canary alert** firing on privileged-tool use: a Web Bug Canarytoken triggered by a `kubeconfig_get` invocation, with a real source IP and a client string identifying the decoy server and tool (`mcp-deception-incubator-kubernetes/1.0 (tool=kubeconfig_get; ...)`).

This is the validated basis for the "registry-layer" claim: real and decoy MCP servers behind one portal, an agent treating them as equally valid, and the two-stage detection firing on privileged use.

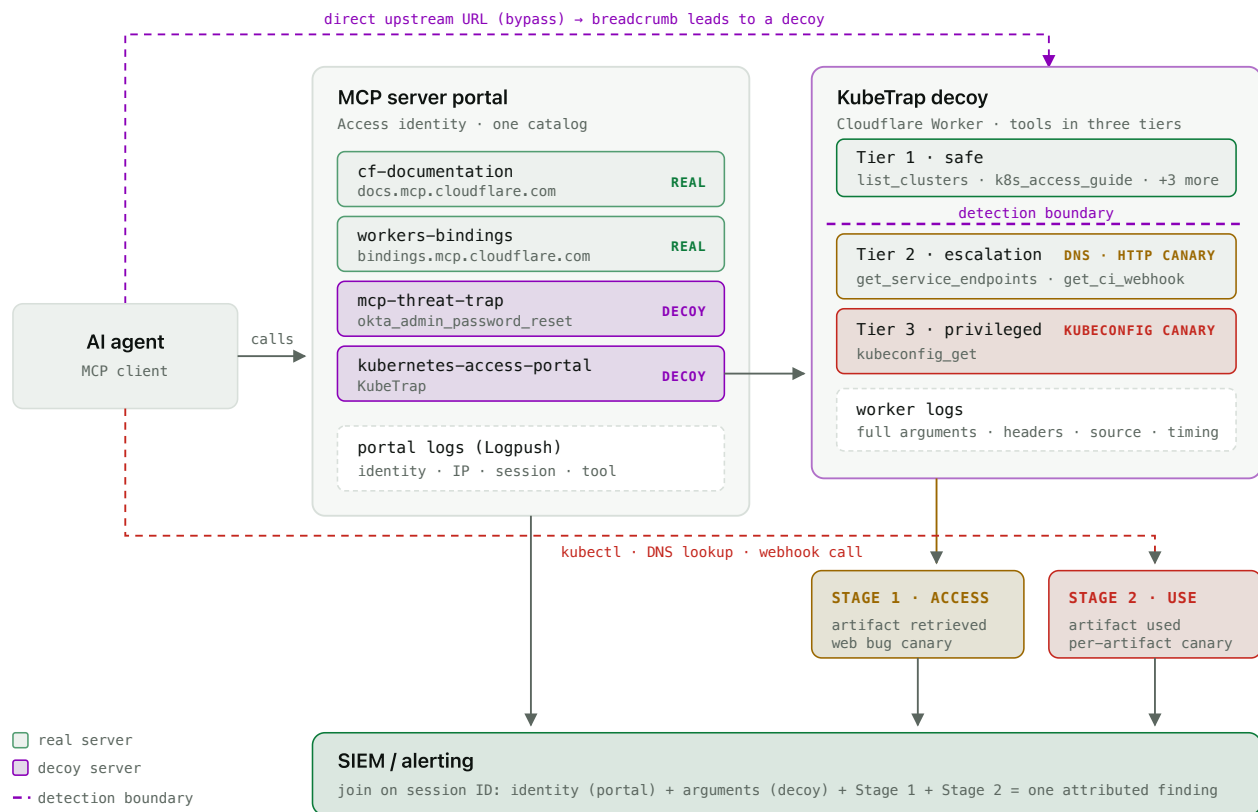


Fig. 1 — Registry-layer deception architecture: an enumerating agent reaches a gateway portal exposing two real Cloudflare-hosted MCP servers and two decoy servers side by side; within a decoy server, tools are tiered from safe to privileged, with canaries embedded in every tool past the detection boundary that fire in two stages — access and use — into a SIEM.

(OWASP demonstration recording, published by the OWASP Foundation: <https://youtu.be/amS0icidXOs>.)

9. Prior-art timeline

This record establishes the chronology of the work relative to subsequent, independently developed approaches in the field.

- **May 21, 2025** — Initial public release of *MCP Threat Trap* (github.com/harshadk99/deception-remote-mcp-server, MIT license): a serverless MCP deception honeypot on Cloudflare Workers that triggers a Canarytoken when a decoy internal admin tool is invoked. Earliest public timestamp for this detection methodology.
- **2025 conference circuit** — Presented via competitive/blind CFP review at BSides Orlando (Sept 2025; recording published by BSides Orlando: <https://youtu.be/aleUOMi73fA>), DevSecCon 2025 (recording: <https://youtu.be/mXiTIMMByxE>), BSides Chicago (Nov 2025), and others.
- **January 29, 2026** — *MCP Threat Trap* demonstrated in a public livestream for the AI Security Engineers Community (<https://youtu.be/DEAI84LkngQ>).

- **February 2026** — *MCP Deception Incubator* presented at the OWASP 25th Anniversary Conference, including the registry-layer gateway with real and decoy MCP servers side by side and two-stage detection (this paper's architecture), demonstrated live.
- **June 13, 2026** — *KubeTrap* (Kubernetes MCP Trap) demonstrated at BSides Boulder, with the three-tier / two-stage architecture and a live firing.
- **June 22, 2026** — Thinkst, an arms-length deception vendor (Canary / Canarytokens), publicly cited the author's open-source AI honeypot work in the release of its MCP Canarytoken, linking to github.com/harshadk99/mcp-deception-incubator-kubernetes: "Using a similar alerting model, Harshad Sadashiv Kadam used our Kubernetes, DNS and Web Bug tokens to create an AI honeypot that would alert only after a client started using the results from tool calls." The post contrasts its own approach as requiring no infrastructure — a single `mcp.json` entry.
- **September 24, 2026** — Cloudflare announced general availability of MCP server portals, the reference implementation for this architecture (Section 4). The February 2026 demonstration of real and decoy servers in a single portal predates general availability by seven months.

(Sources for each entry are listed under References.)

10. Related work and field context

The proposition that deception is the right control against autonomous agents is now widely held. The Cloud Security Alliance's May 2026 guidance, *The "AI Vulnerability Storm": Building a "Mythos-ready" Security Program*, lists "Build a Deception Capability" as a high-priority control for the next 90 days — "Deploy canaries and honey tokens"; OpenAI, presenting at Black Hat USA 2026, included deception among what Thinkst described as the "few recommendations" it made; and vendor benchmarking of frontier models against canaries in cloud accounts has reported early warning in most compromising runs (Tracebit, May 2026).

Commercial and open-source single-decoy approaches occupy the token layer and are valuable first moves:

- **Thinkst's MCP Canarytoken** generates decoy `mcp.json` entries that alert on connection or tool call; its **Agent Provocateur** engages and questions attacking agents on the network, and has been observed tripping across both open and frontier models.
- **Tracebit's context bombs** place content in decoy secrets designed to trip an attacking model's own safety guardrails, tested across five frontier models in a purpose-built AWS cyber range.
- **Lenny Zeltser's decoy MCP server guide** documents building a workstation tripwire on Cloudflare Workers.
- **HoneyMCP** injects ghost tools into an MCP server to detect and engage attackers at the single-server level.
- **Decoy Tripwire** is a local proxy that adds decoy tools alongside a client's real servers and pauses the agent when one fires.

- **Praetorian's Knossos** generates realistic decoy AWS environments from an organization's real naming and IAM patterns — a registry-adjacent idea, but for cloud accounts rather than MCP registries.

Each of these places a decoy on an endpoint, inside a single server, in a cloud account, or in front of one client. This paper is complementary: it addresses the layer above the single token, arguing that as agentic attacks become routine, deception must be *structured as an environment* — tiered tools, staged artifacts, and real and decoy servers interleaved at the MCP registry itself — rather than planted as an isolated tripwire. The tripwire tells you someone is there. The architecture tells you what they came to do.

11. Limitations and future work

- **Detection, not prevention.** The architecture surfaces intent; containment and response are downstream and out of scope here.
 - **Depends on enumeration.** Effective against agents that explore by default; a targeted adversary with prior environment knowledge may not enumerate.
 - **Benign agents can trip decoys.** A legitimate agent selects tools autonomously; a decoy description that overlaps with a real task will produce false positives. Descriptions must target capabilities outside every legitimate user's scope, and the visibility policy in Section 7 should narrow exposure where that risk is highest.
 - **Decoys can be fingerprinted.** Differences in hosting, latency, tool count, empty resource lists, or response style can separate decoys from real servers to a careful attacker. Decoys should be hosted and shaped like their neighbors.
 - **Realism is an operational cost.** Decoy servers and tools must remain believable relative to the real servers they sit beside, and this cost recurs — see the monthly review step in Section 7.
 - **Portal capacity is finite.** Cloudflare's portals support a bounded number of servers per portal; decoys consume that budget alongside real ones.
 - **Sync and expiry apply to decoys too.** The same fixed-interval sync and silent admin-credential expiry described in Section 4.5 affect decoy servers exactly as they affect real ones, and an expired decoy fails silently rather than loudly.
 - **Guardrail-dependent responses may not hold.** A response strategy that relies on the attacking model's own safety training (Section 6.7) assumes those guardrails are present; a real attacker may not be using a model that has them. Treat anything an agent reports about itself as unverified — see the hallucinated-SSH-key example in Section 3.
 - **Scope.** This paper covers remote HTTP MCP servers behind a gateway portal. Local stdio servers are out of scope, though the principles should transfer.
 - **Future directions.** Agent-to-agent (A2A) deception surfaces; adaptive decoys that evolve with the environment; measurement of evasion cost when decoys are interleaved with real servers versus deployed standalone; standardized telemetry for the two-stage chain into common SOC pipelines.
-

12. Conclusion

The debate over whether deception works against autonomous AI agents is settled; it works. The next question is structural. A single decoy catches curiosity but can be avoided and cannot cleanly separate reconnaissance from intent. Making the registry the unit of deception — real and decoy MCP servers side by side behind one gateway, with a two-stage boundary that distinguishes finding a credential from using it — turns an agent's own default behavior, exhaustive enumeration, into a high-fidelity signal. Early cloud-account evidence suggests that signal may grow stronger as models get more capable (Sections 2 and 13). This architecture was built on zero standing infrastructure and demonstrated live in February 2026. It is offered here, openly, as a reference for teams building detection for AI-native environments.

13. Next

Existing empirical work on agents and deception measures canaries placed in cloud accounts (Section 2). It does not measure the setting this paper describes: real and decoy MCP servers presented side by side in one registry, where the agent chooses among them rather than stumbling on a planted secret. Three questions follow directly from the architecture:

1. Do agents distinguish decoy servers from real ones in a shared catalog, and does that ability grow with model capability?
2. How often does an agent that crosses Stage 1 (access) go on to Stage 2 (use), and does that ratio vary by model?
3. Does the capability inversion observed at the cloud-account level — more capable models tripping more decoys — hold at the registry layer?

At OWASP LASCON 2026 (Austin, October 29–30), the author will present benchmark results measuring how different frontier models behave when they discover real and decoy servers side by side on an MCP portal — the empirical follow-up to the architecture presented here.

References

All sources are public. Links were checked on September 25, 2026.

MCP server portals (Sections 4–6, 9)

1. Cloudflare, "MCP server portals," Cloudflare One documentation, updated Sept. 24, 2026.
<https://developers.cloudflare.com/cloudflare-one/access-controls/ai-controls/mcp-portals/>
2. Cloudflare, "MCP server portals now generally available," changelog, Sept. 24, 2026 — GA status, Code Mode, service-token authentication, Logpush.
<https://developers.cloudflare.com/changelog/post/2026-09-24-mcp-portals-ga/>

3. Cloudflare, "MCP portal logs," Logpush dataset reference — exported fields; no call arguments (Sections 4.5, 6.3). https://developers.cloudflare.com/logs/logpush/logpush-job/datasets/account/mcp_portal_logs/
4. Cloudflare, "How Cloudflare detects MCP traffic and helps secure it," Aug. 18, 2026 — Gateway MCP traffic detection (Section 6.5). <https://blog.cloudflare.com/mcp-security-updates/>

Threat model (Section 5)

5. Microsoft, "The state of MCP security in 2026," June 30, 2026 — the confused-deputy pattern. <https://techcommunity.microsoft.com/blog/microsoft-security-blog/the-state-of-mcp-security-in-2026/4531327>
6. Cloud Security Alliance, "MCP Tool Poisoning: Adversarial Hijacking of AI Agent Workflows," July 2, 2026 — tool poisoning and rug-pull attacks. <https://labs.cloudsecurityalliance.org/research/csa-research-note-mcp-tool-poisoning-ai-agent-exfiltration-2/>

Field context and related work (Sections 2, 3, 10)

7. Tracebit Research, "AI Agents & Canaries: deception warns your teams at the speed of an AI attacker," v1.0, May 28, 2026 — ten frontier models in a controlled AWS cyber range (Sections 2, 10). <https://agentic.tracebit.com/>
8. Cloud Security Alliance, "The 'AI Vulnerability Storm': Building a 'Mythos-ready' Security Program," May 1, 2026 — "Build a Deception Capability." <https://cloudsecurityalliance.org/artifacts/the-ai-vulnerability-storm-building-a-mythos-ready-security-program>
9. Thinkst (Haroon Meer), "Getting Agents to tell on themselves," Sept. 4, 2026 — Agent Provocateur, and OpenAI's deception recommendation at Black Hat USA 2026 (Sections 3, 10). <https://blog.thinkst.com/2026/09/getting-agents-to-tell-on-themselves.html>
10. OpenAI, "The 'Breaking' News: The OpenAI–Hugging Face Incident," Black Hat USA 2026, published Aug. 6, 2026. <https://www.youtube.com/watch?v=87DyyMV0kCY>
11. Thinkst (Jacob Torrey), "One for all the models out there!" June 22, 2026 — the MCP Canarytoken (Sections 9, 10). <https://blog.thinkst.com/2026/06/one-for-all-the-models-out-there.html>
12. Tracebit Research, "Context Bombs: stopping AI attackers in their tracks," July 12, 2026. <https://agentic.tracebit.com/context-bombs/>
13. Lenny Zeltser, "Build a Decoy MCP Server to Catch AI Agent Attackers," May 2026. <https://zeltser.com/decoy-mcp-server-honeypot>
14. HoneyMCP. <https://github.com/barvhaim/HoneyMCP>
15. Decoy Tripwire (`decoy-tripwire`), first published Mar. 27, 2026. <https://github.com/decoy-run/decoy-tripwire>
16. Praetorian (Mario Bartolome), "Knossos: Procedurally Generated Decoy Environments," July 2, 2026. <https://www.praetorian.com/blog/knossos-decoy-environments>

The author's work (Sections 8, 9)

17. MCP Threat Trap, first released May 21, 2025. <https://github.com/harshadk99/deception-remote-mcp-server>

18. KubeTrap (MCP Deception Incubator, Kubernetes). <https://github.com/harshadk99/mcp-deception-incubator-kubernetes>
 19. "MCP Deception Incubator — Honeytraps as a Framework for Zero Trust AI Environments," OWASP 25th Anniversary Conference, Feb. 2026. Recording published by the OWASP Foundation:
<https://youtu.be/amS0icidxOs> · session page:
<https://owasp25thanniversaryvirtual.sched.com/event/2Djni/mcp-deception-incubator-honeytraps-as-a-framework-for-zero-trust-ai-environments-track-2>
-

Corresponding work and source repositories: github.com/harshadk99